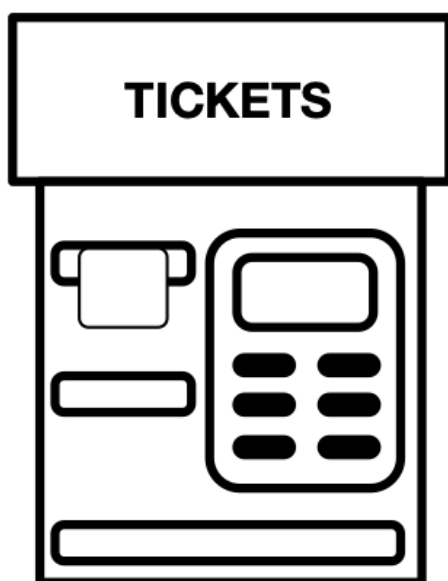




instituto
superior de
engenharia
de lisboa

Licenciatura em Engenharia Informática e de Computadores



MÁQUINA DE VENDA DE BILHETES (*Ticket Machine*)

Projeto
de
Laboratório de Informática e Computadores
2025 / 2026 verão

Tiago Inês, Tiago Pedroso, Tomás Ferreira

05 de julho de 2026

Conteúdo

1	Descrição	1
2	Arquitetura do Sistema	3
2.1	<i>Keyboard Reader</i>	3
2.1.1	<i>Ring Buffer</i>	3
2.2	<i>Coin Acceptor</i>	4
2.3	<i>Port Expander LCD</i>	5
2.4	<i>Port Expander Ticket Dispenser</i>	5
2.5	<i>Control</i>	6
2.5.1	Protocolo de comunicação série (<i>software</i>)	6
3	Resultados Experimentais	8
4	Conclusões	9
5	Trabalho Futuro	9
	Referências	11
A	Ticket Machine - Hardware	12
A.1	TicketMachine.vhd	12
A.2	Atribuição de Pinos	16
A.3	Atribuição do USBPort	19
B	Ticket Machine - Software (<i>Control</i>)	20
B.1	Ticket Machine - App	20
B.2	TUI	28
B.3	Coin Acceptor	36
B.4	Coin Deposit	37
B.5	Stations	38
B.6	File Access	38

Lista de Figuras

1	Diagrama de blocos da Máquina de Venda de Bilhetes (<i>Ticket Machine</i>)	1
2	Arquitetura do sistema que implementa a máquina de venda de bilhetes (<i>Ticket Machine</i>)	3
3	Diagrama de blocos do <i>Keyboard Reader</i>	4
4	Bloco <i>Coin Acceptor</i>	5
a	Diagrama de blocos	5
b	Diagrama temporal	5
5	Diagrama de blocos do módulo <i>Port Expander LCD (PELCD)</i>	5
6	Diagrama de blocos do módulo <i>Port Expander Ticket Dispenser (PETD)</i>	6
7	Diagrama lógico da Máquina de Venda de Bilhetes (<i>Ticket Machine</i>)	6

Lista de Tabelas

1	Codificação do valor facial das moedas	5
2	Atribuição de pinos do ficheiro TicketMachine.vhd [4]	17
3	Atribuição de variáveis para o USBPort	19

Lista de Algoritmos

1	Ticket Machine Main Module	12
2	Ticket Machine Application	20
3	TUI	28
4	Coin Acceptor	36
5	Coin Deposit	37
6	Stations	38
7	File Access	38

1 Descrição

Este projeto implementa uma máquina de venda de bilhetes (*Ticket Machine*), que permite a aquisição de bilhetes de comboio. O percurso é definido pela estação de origem, local de compra do bilhete e pela seleção do destino digitando o identificador da estação ou através das teclas ↑ e ↓, sendo exibido no ecrã além do identificador da estação de destino o preço e o tipo de bilhete (ida ou ida/volta). A ordem de aquisição é dada através da pressão da tecla de confirmação, sendo impressa uma unidade do bilhete exibido no ecrã. A máquina não realiza trocos e só aceita moedas de: 0,05€; 0,10€; 0,20€; 0,50€; 1,00€; e 2,00€. Para além do modo de Dispensa, o sistema tem mais um modo de funcionamento designado por Manutenção, que é ativado por uma chave de manutenção. Este modo permite o teste da máquina de venda de bilhetes, bem como permite iniciar e consultar os contadores de bilhetes e moedas. A máquina de venda de bilhetes é constituída pelo sistema de gestão (designado por *Control* na Figura 1) e pelos seguintes periféricos: um teclado de 16 teclas, um moedeiro (designado por *Coin Acceptor*), um ecrã *Liquid Cristal Display* (LCD) de duas linhas de 16 caracteres, um mecanismo de dispensa de bilhetes (designado por *Ticket Dispenser*) e uma chave de manutenção (designada por *M*) que define se a máquina de venda de bilhetes está em modo de Manutenção, conforme o diagrama de blocos apresentado na Figura 1.

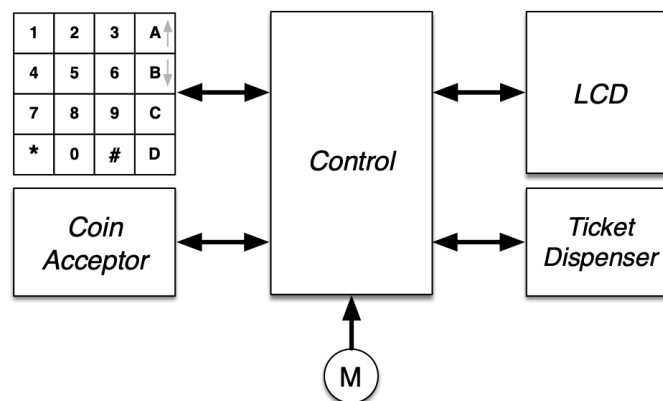


Figura 1: Diagrama de blocos da Máquina de Venda de Bilhetes (*Ticket Machine*)

Sobre o sistema podem-se realizar as seguintes ações em modo Venda:

- **Consulta e venda** A consulta de um bilhete é realizada digitando o identificador da estação de destino ou listando-a através das teclas ↑ e ↓. O processo de compra do bilhete inicia-se premindo a tecla '#'. Durante a inserção do respetivo valor monetário, é possível alterar o tipo de bilhete (ida ou ida/volta) premindo a tecla '*', com a consequente alteração do preço da viagem afixado no *LCD*, duplicando o valor no caso de ida/volta. Durante a compra ficam afixados no *LCD* as informações referentes ao bilhete pretendido até que o mecanismo de impressão de bilhetes confirme que a impressão já foi realizada e a recolha do bilhete efetuada. A compra pode ser cancelada premindo a tecla '#', devolvendo as moedas inseridas.

Sobre o sistema podem-se realizar as seguintes ações em modo Manutenção:

- **Teste** – Esta opção do menu permite realizar um procedimento de consulta e venda de um bilhete, sem introdução de moedas e sem esta operação ser contabilizada como uma aquisição.
- **Consulta Bilhetes** – Premindo a tecla 'A', em seguida através das teclas ↑ e ↓, visualizam os contadores do número de bilhetes vendidos.
- **Consulta Moedas** – Premindo a tecla 'B', em seguida através das teclas ↑ e ↓, visualizam os contadores das moedas no cofre.
- **Iniciar os contadores de moedas e bilhetes** – Premindo a tecla 'C' e em seguida a tecla '*', o sistema de gestão coloca os contadores de moedas e bilhetes a zero, iniciando um novo ciclo de contagem.
- **Desligar** – Premindo a tecla 'D' permite desligar o sistema, que encerra apenas após a confirmação do utilizador, ou seja, o programa termina e as estruturas de dados, contendo a informação dos contadores de moedas e da lista de bilhetes vendidos, são armazenadas de forma persistente em dois ficheiros de texto. A informação em cada ficheiros deve estar organizada por linha, em que os campos de dados são separados por “;”, com o respetivo formato: 'COIN_VALUE;NUMBER_COINS' (valor facial; número de moedas) e 'PRICE;SOLD_TICKETS;STATION_NAME' (preço; número de bilhetes vendidos para a estação; nome da estação). Os dois ficheiros devem ser carregados para o sistema no seu processo de arranque.

Nota: A inserção de informação através do teclado tem o seguinte critério: *i*) se não for premida nenhuma tecla num intervalo de cinco segundos o comando em curso é abortado; *ii*) quando o dado a introduzir é composto por mais que um dígito, são considerados apenas os últimos dígitos, a inserção realiza-se do dígito de maior peso para o de menor peso.

2 Arquitetura do Sistema

O sistema implementa uma solução híbrida de hardware e software, como apresentado no diagrama de blocos da Figura 2. A arquitetura proposta é constituída por cinco módulos principais: *i*) um leitor de teclado, designado por *Keyboard Reader*; *ii*) um módulo de interface com o *LCD*, designado por *Port Expander LCD (PELCD)*; *iii*) um módulo de interface com o mecanismo de dispensa de bilhetes (*Ticket Dispenser*), designado por *Port Expander Ticket Dispenser (PETD)*; *iv*) um moedeiro, designado por *Coin Acceptor*; e *v*) um módulo de controlo, designado por *Control*.

Os módulos *i*), *ii*) e *iii*) deverão ser implementados em hardware, o moedeiro (*iv*) será emulado utilizando quatro interruptores e três *LED*, enquanto o módulo de controlo (*v*) deverá ser implementado em software, descrito em linguagem *Kotlin* e executado num PC.

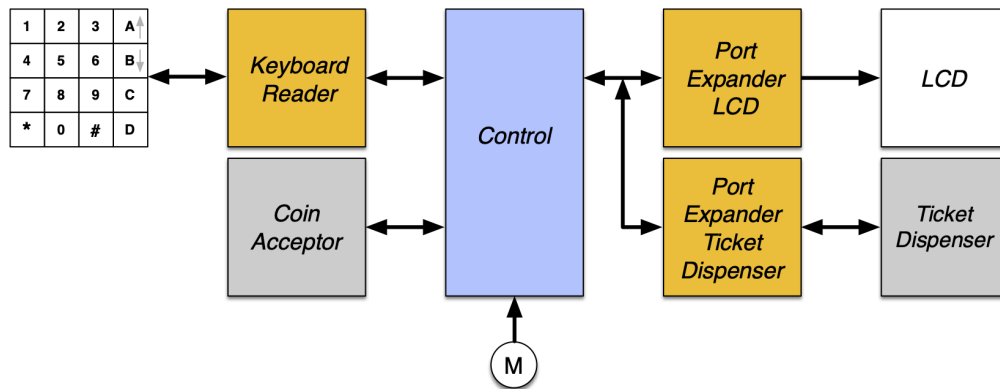


Figura 2: Arquitetura do sistema que implementa a máquina de venda de bilhetes (*Ticket Machine*)

O módulo *Keyboard Reader* é responsável pela descodificação do teclado matricial de 16 teclas, determinando qual a tecla pressionada e disponibilizando o seu código, com quatro bits, ao módulo *Control*. Caso este não esteja disponível para o receber imediatamente, o código da tecla é armazenado internamente, até ao limite de dezoito (18) códigos. O módulo *Control* processa os dados e envia a informação a apresentar no *LCD* através do módulo *PELCD*. O *Ticket Dispenser* é atuado pelo módulo *Control*, através do módulo *PETD*. Por forma a minimizar o número de interligações, a comunicação entre o módulo *Control* e os módulos *PELCD* e *PETD* é realizada através de um protocolo série.

2.1 *Keyboard Reader*

O módulo *Keyboard Reader* [1] é constituído por três blocos principais: *i*) o decodificador de teclado (*Key Decode*); *ii*) o bloco de armazenamento (designado por *Ring Buffer*); e *iii*) o bloco de entrega ao consumidor (designado por *Key Transmitter*). Neste caso o módulo *Control*, implementado em software, é a entidade consumidora.

2.1.1 *Ring Buffer*

O bloco *Ring Buffer* implementa uma fila FIFO de 16 posições de 4 bits, responsável por desacoplar o ritmo de produção de teclas (lado do *Key Decode*) do ritmo de consumo (lado do *Key*

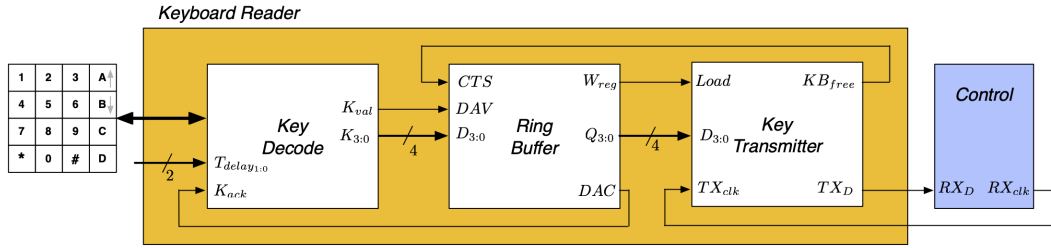


Figura 3: Diagrama de blocos do *Keyboard Reader*

Transmitter). A sua arquitectura, representada na Figura 3, é composta por três sub-blocos:

- **RAM** — memória de 16 palavras de 4 bits, construída estruturalmente a partir de 16 registos **Reg** endereçados por um decodificador **DecoderBuffer**. A leitura é combinatória; a escrita é síncrona e controlada pelo sinal **wr**.
- **MAC** (*Memory Address Control*) — mantém dois apontadores de 4 bits, **putIndex** (escrita) e **getIndex** (leitura), cada um implementado com um somador e um registo. Um sub-bloco **RAMEF** conta o número de elementos presentes e gera os sinais **full** e **empty**. O sinal **putNget** selecciona, via **MUX4**, qual dos dois apontadores é apresentado à RAM em cada ciclo.
- **RBC** (*Ring Buffer Control*) — máquina de estados que arbitra o acesso à RAM entre as operações de escrita (proveniente do *Key Decode* via **DAV**) e de leitura (solicitada pelo *Key Transmitter* via **CTS**). A escrita é confirmada ao produtor através do sinal **DAC**; a leitura é sinalizada ao *Key Transmitter* através do sinal **Wreg**.

O protocolo de escrita segue o handshake **DAV/DAC** definido no enunciado: o *Key Decode* activa **DAV** e mantém os dados estáveis; o *Ring Buffer* escreve em memória, activa **DAC** e aguarda que **DAV** baixe antes de regressar ao estado de espera. A leitura é iniciada pelo *Key Transmitter* através de **CTS**: o *Ring Buffer* coloca os dados de **getIndex** na saída **Q**, pulsa **Wreg** para que o *Key Transmitter* os latche, e incrementa **getIndex**.

2.2 Coin Acceptor

O módulo *Coin Acceptor* implementa a interface com o moedeiro, sinalizando ao módulo *Control* que o moedeiro recebeu uma moeda através da ativação do sinal *Coin*, o valor monetário da moeda inserida é codificado em $Coin_{id_{2:0}}$, conforme a codificação apresentada na Tabela 1. A entidade consumidora informa o *Coin Acceptor* que já contabilizou a moeda ativando o sinal *accept*, conforme apresentado no diagrama temporal da Figura 4b. O *Control* pode devolver as moedas inseridas no moedeiro através da ativação do sinal *eject* durante dois segundos, ou recolher as moedas presentes no moedeiro através da ativação do sinal *collect* durante dois segundos.

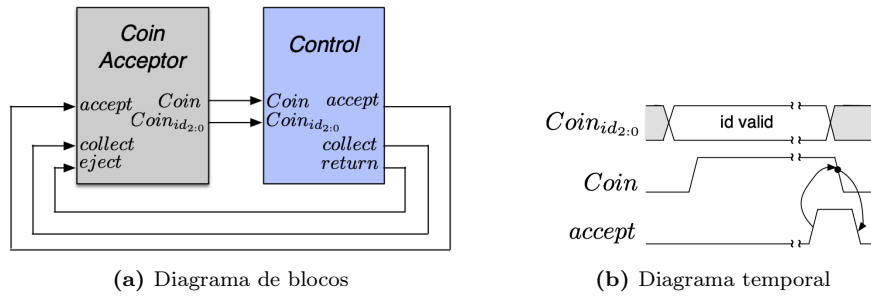


Figura 4: Bloco *Coin Acceptor*

Tabela 1: Codificação do valor facial das moedas

Moeda [€]	Codificação [$Cid_2Cid_1Cid_0$]
0,05	000
0,10	001
0,20	010
0,50	011
1,00	100
2,00	101

2.3 Port Expander LCD

O módulo *Port Expander LCD* [2] (*PELCD*) implementa a interface com o *LCD*, fazendo a receção em série da informação enviada pelo módulo de controlo e entregando-a posteriormente ao *LCD*, conforme representado na Figura 5.

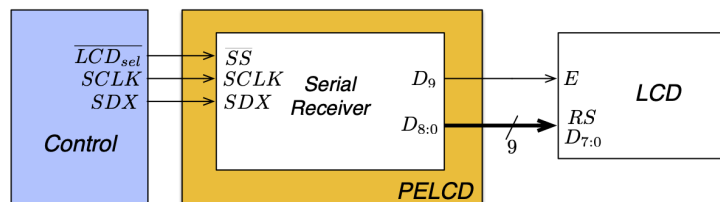


Figura 5: Diagrama de blocos do módulo *Port Expander LCD* (*PELCD*)

2.4 Port Expander Ticket Dispenser

O módulo *Port Expander Ticket Dispenser* [3] (*PETD*) implementa a interface com o mecanismo de impressão de bilhetes (*Ticket Dispenser*), realizando a receção em série da informação enviada pelo módulo de controlo e entregando-a posteriormente ao mecanismo, conforme representado na Figura 6.

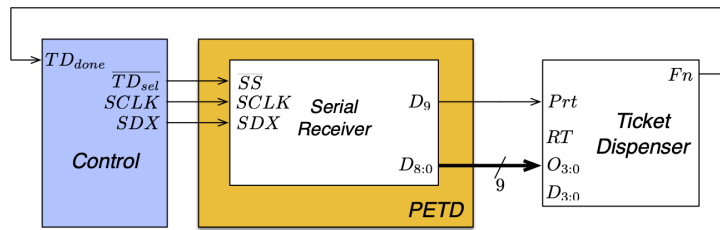


Figura 6: Diagrama de blocos do módulo *Port Expander Ticket Dispenser* (PETD)

2.5 Control

A implementação do módulo *Control* foi realizada em software, usando a linguagem *Kotlin* e seguindo a arquitetura lógica apresentada na Figura 7.

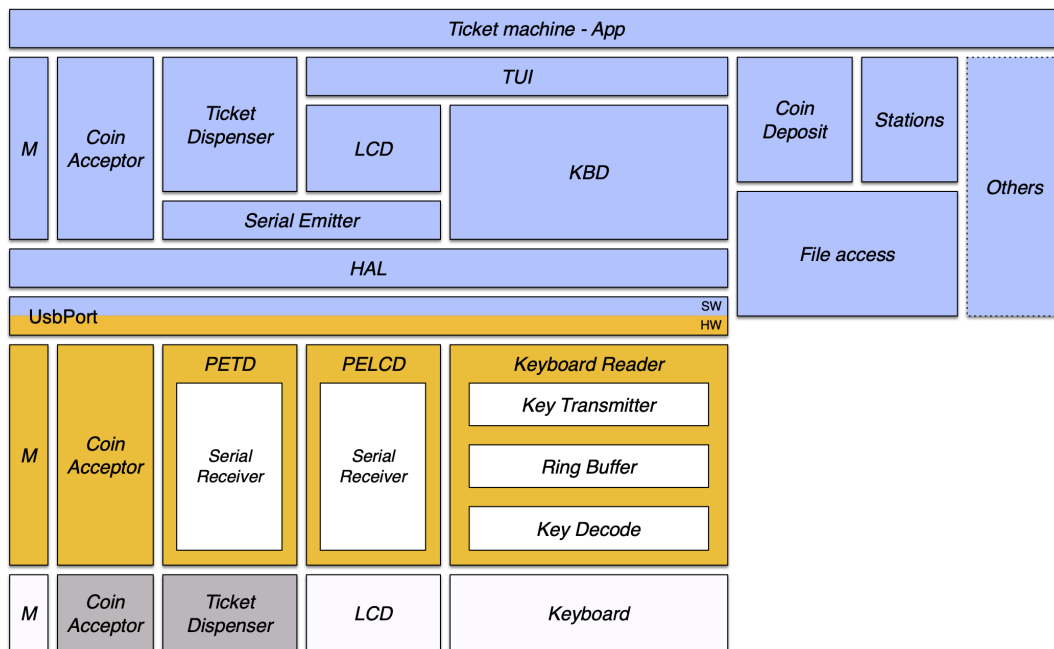


Figura 7: Diagrama lógico da Máquina de Venda de Bilhetes (*Ticket Machine*)

2.5.1 Protocolo de comunicação série (*software*)

A comunicação entre o módulo *Control* e os módulos *PELCD* e *PETD* é realizada pelo objecto *SerialEmitter*, que partilha as linhas *SDX* (dados) e *SCLK* (relógio) entre os dois periféricos, diferenciando-os através de sinais de selecção independentes: *SSLCD* (bit 2 do *output port*) para o *PELCD* e *SSTD* (bit 3) para o *PETD*.

O procedimento de envio de uma trama de n bits é o seguinte:

1. Desactivar *SCLK* e activar o *slave select* do periférico de destino (transição descendente em $\overline{SS}$), assinalando o início de trama.
2. Para cada bit, do LSB para o MSB: colocar o bit em *SDX*, aguardar a estabilização do sinal, activar *SCLK* e desactivar *SCLK*. O *Serial Receiver* no hardware amostra *SDX* na transição

ascendente de $SCLK$.

3. Activar $\overline{SS}$ (transição ascendente), gerando a condição de fim de trama. Neste momento o *Hold Register* do *Serial Receiver* captura o conteúdo do *Shift Register*, tornando os dados disponíveis nos periféricos.
4. Emitir um pulso de relógio adicional após a subida de $\overline{SS}$ para garantir que o *Hold Register* capta correctamente os dados antes que SDX mude de estado.

A trama enviada para o *PELCD* tem 10 bits no formato $[RS, D_0, D_1, \dots, D_7, E]$, sendo RS o primeiro bit transmitido. A trama enviada para o *PETD* tem igualmente 10 bits no formato $[RT, O_0, O_1, O_2, O_3, D_0, D_1, D_2, D_3, Prt]$, onde RT indica o tipo de bilhete (ida/ida e volta), $O_{3:0}$ e $D_{3:0}$ são os códigos da estação de origem e de destino, e Prt activa a impressão.

O mapeamento completo entre sinais lógicos e bits do *UsbPort* está detalhado na Tabela 3 do Apêndice.

3 Resultados Experimentais

A verificação do sistema foi realizada em duas fases distintas: simulação dos módulos de *hardware* com recurso ao *ModelSim* e teste do sistema completo na placa de desenvolvimento *DE10-Lite*, com o módulo de controlo a correr num PC via ligação *USB/JTAG*.

Simulação dos módulos de *hardware*

Todos os módulos *VHDL* desenvolvidos foram verificados individualmente através de *testbenches* dedicados. Os módulos validados incluem os blocos combinatórios e sequenciais da biblioteca comum (FFD, Reg, RAM, AdderSubtrator, CLKDIV), o *Keyboard Reader* completo (Key_Scan, Key_Decode, KeyDecoderFSM, KeyTransmitter, RingBuffer), o módulo *PELCD* com o respectivo *SerialReceiver*, e o *PETD* com o *TICKET_DISPENSER*.

Os resultados das simulações confirmaram o funcionamento correcto dos protocolos série implementados: a recepção de tramas de 10 bits no *PELCD* e no *PETD* produziu os valores esperados nos registos de saída, e o *Ring Buffer* respeitou a disciplina FIFO com as condições de *full/empty* correctamente assinaladas.

Testes na placa

O sistema integrado foi testado na placa *DE10-Lite* em modo de venda e em modo de manutenção. As funcionalidades verificadas incluem:

- Consulta e navegação de destinos através das teclas $\uparrow$ e $\downarrow$ e por digitação directa do identificador;
- Inserção de moedas, actualização do montante em falta no *LCD* e dispensa de bilhete com activação do sinal *collect*;
- Cancelamento de compra com devolução das moedas via sinal *eject*;
- Acesso ao modo de Manutenção pela chave *M* e utilização de todas as opções do menu (Teste, Consulta Bilhetes, Consulta Moedas, Reset e Desligar);
- Persistência de dados: os contadores de moedas e bilhetes são correctamente guardados nos ficheiros *CoinDeposit.txt* e *stations.txt* e recarregados no arranque seguinte.

4 Conclusões

O trabalho consistiu no desenvolvimento de uma máquina de venda de bilhetes de comboio, implementada numa solução híbrida de *hardware* e *software*. A componente de *hardware* foi descrita em *VHDL* e sintetizada na placa *DE10-Lite* (FPGA MAX 10), englobando o *Keyboard Reader*, os módulos *PELCD* e *PETD*, o *Ticket Dispenser* e o *Coin Acceptor*. A componente de *software* foi desenvolvida em *Kotlin* e executada num PC, comunicando com o *hardware* através da porta virtual *JTAG/USB*.

Os resultados experimentais confirmam o funcionamento do sistema em ambos os modos de operação definidos no enunciado — modo de Venda e modo de Manutenção — com excepção da anomalia esporádica no leitor de teclado descrita na secção anterior.

Do ponto de vista das aprendizagens, o projecto foi particularmente relevante em dois domínios:

Integração *hardware/software*. A necessidade de coordenar sinais com semântica de nível (*Kval*) com protocolos orientados a pulsos (*Load/KBfree*) evidenciou a importância de uma especificação rigorosa das interfaces entre os dois domínios. A introdução do módulo *KeyInterface*, que resolve o *deadlock* de *handshake* entre o *KeyDecode* e o *KeyTransmitter*, ilustra como problemas de integração surgem frequentemente nas fronteiras entre blocos, mesmo quando cada bloco individualmente está correcto.

Arquitectura em camadas (*HAL/TUI*). A separação entre a camada de abstracção de *hardware* (*HAL*), a camada de interface série (*SerialEmitter*), os periféricos lógicos (*LCD*, *KBD*, *CoinAcceptor*, *TicketDispenser*) e a lógica aplicacional (*App*) permitiu desenvolver e testar cada módulo de forma independente, reduzindo significativamente o esforço de depuração na fase de integração. Esta organização, semelhante à de sistemas embebidos reais, demonstrou o valor prático de uma arquitectura em camadas mesmo em projectos de dimensão académica.

5 Trabalho Futuro

No âmbito do desenvolvimento futuro da máquina de venda de bilhetes, existem diversas melhorias que poderiam ser implementadas de forma a enriquecer a experiência do utilizador e aumentar a funcionalidade do sistema.

Uma das melhorias mais relevantes seria a implementação de um sistema de histórico das últimas compras realizadas. Esta funcionalidade permitiria que no modo de manutenção podessemos ver o histórico das últimas viagens, mostrando informações como a estação de destino, o tipo de bilhete adquirido (ida ou ida/volta) e o valor pago. Este histórico poderia ser armazenado de forma persistente num ficheiro de texto, seguindo a mesma abordagem já utilizada para os contadores de moedas e bilhetes, sendo carregado no arranque do sistema e atualizado a cada nova compra.

Outra melhoria significativa seria a implementação de um sistema de troco automático. Actualmente, a máquina não realiza trocos, o que obriga o utilizador a inserir o valor exato do bilhete. Com esta funcionalidade, o sistema calcularia automaticamente a diferença entre o valor inserido e o preço do bilhete, devolvendo o troco ao utilizador através do mecanismo de ejeção de moedas.

já existente.

Por fim, uma melhoria igualmente importante seria o suporte a notas, nomeadamente notas de 5€ e 10€. Dado que no trabalho não utilizamos o espaço da lista do coinId referente ao 110 e ao 111 nos switches SW2, SW3 e SW4 respetivamente, seria apenas possível adicionar mais estas duas notas.

Referências

- [1] David Velez, Nuno Sebastião, Pedro Miguens Matutino, Rogério Rebelo, Rui Duarte *KeyBoardReader datasheet*, 2026, ISEL.
- [2] David Velez, Nuno Sebastião, Pedro Miguens Matutino, Rogério Rebelo, Rui Duarte *Port Expander LCD datasheet*, 2026, ISEL.
- [3] David Velez, Nuno Sebastião, Pedro Miguens Matutino, Rogério Rebelo, Rui Duarte *Port Expander Ticket Dispenser datasheet*, 2026, ISEL.
- [4] Terasic *DE10-lite User Manual*, 2020, Terasic.
- [5] Terasic *de10-Lite Exp. Board - Reference Manual*, 2022, Pedro Miguens Matutino
- [6] David Velez, Nuno Sebastião, Pedro Miguens Matutino, Rogério Rebelo, Rui Duarte *Enunciado TicketMachine*, 2026, ISEL.
- [7] David Velez, Nuno Sebastião, Pedro Miguens Matutino, Rogério Rebelo, Rui Duarte *Moodle LIC*, 2026, ISEL.

A Ticket Machine - Hardware

A.1 TicketMachine.vhd

O ficheiro `TicketMachine.vhd` constitui o módulo de topo (*top-level*) do sistema de hardware. A sua função é exclusivamente estrutural: instancia todos os módulos de hardware e estabelece as ligações entre eles e com os pinos físicos da placa *DE10-Lite*.

Os módulos instanciados e as suas interligações principais são os seguintes:

- **UsbPort** — ponte virtual *JTAG* que expõe um *input port* de 8 bits (lido pelo *software*) e um *output port* de 8 bits (escrito pelo *software*). O mapeamento de bits está descrito na Tabela 3.
- **KeyboardReader** — recebe as linhas do teclado matricial (`Keys_Vertical/Keys_Horizontal`), o relógio de transmissão série `TXclk` (bit 7 do *output port*) e expõe `TXD` (bit 7 do *input port*) com o código da tecla serializado.
- **PELCD** — recebe `SDX` (bit 0), `SCLK` (bit 1) e `SSLCD` (bit 2) do *output port*; a sua saída de 10 bits é desdobrada directamente nos sinais `LCD_RS`, `LCD_DATA[7:0]` e `LCD_EN` ligados ao LCD da placa.
- **PETD** — partilha `SDX` e `SCLK` com o *PELCD*; o seu *slave select* é `SSTD` (bit 3). A saída de 10 bits é mapeada em `RT`, `D[7:0]` e `Prt`, que alimentam o `TICKET_DISPENSER`.
- **TICKET_DISPENSER** — recebe `RT`, `Prt`, os nibbles de origem (`O[3:0]`) e destino (`D[3:0]`) e o sinal de recolha do bilhete `CollectTicket` (interruptor SW); produz o sinal `Fn` (bit 4 do *input port*) e conduz os seis displays de 7 segmentos `HEX0–HEX5`.
- **CoinAcceptor** — lê os interruptores `Coin` e `CId[2:0]` e os comandos `accept/eject/collect` dos bits 4, 5 e 6 do *output port*; acciona os LEDs homónimos na placa.

O sinal de manutenção `M` (interruptor SW, bit 6 do *input port*) é ligado directamente ao *input port* sem qualquer lógica adicional em hardware, sendo interpretado exclusivamente pelo *software*.

Algoritmo 1: Ticket Machine Main Module

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity TicketMachine is
5      port (
6          CLK                : in  std_logic;
7          RESET              : in  std_logic;
8
9          Keys_Vertical      : out std_logic_vector(3 downto 0);
10         Keys_Horizontal    : in  std_logic_vector(3 downto 0);
11         Tdelay              : in  std_logic_vector(1 downto 0);
12
13         LCD_RS              : out std_logic;
14         LCD_EN              : out std_logic;
15         LCD_DATA            : out std_logic_vector(7 downto 0);
16
17         CollectTicket       : in  std_logic;
18

```

```

19         Coin                : in  std_logic;
20         CId                  : in  std_logic_vector(2 downto 0);
21
22         Accept                : out std_logic;
23         Eject                 : out std_logic;
24         Collect               : out std_logic;
25
26         M                     : in  std_logic;
27
28         HEX0, HEX1, HEX2, HEX3, HEX4, HEX5 : out std_logic_vector(7 downto 0)
29     );
30 end entity TicketMachine;
31
32 architecture logicFunction of TicketMachine is
33
34     —————
35     — Component declarations
36     —————
37
38     component KeyboardReader is
39     port (
40         CLK                : in  std_logic;
41         RESET              : in  std_logic;
42         Tdelay              : in  std_logic_vector(1 downto 0);
43         Keys_Vertical       : out std_logic_vector(3 downto 0);
44         Keys_Horizontal     : in  std_logic_vector(3 downto 0);
45         TXclk               : in  std_logic;
46         TXD, clk_out, clk_out_rise : out std_logic
47     );
48 end component KeyboardReader;
49
50     component UsbPort is
51     port (
52         inputPort : in  std_logic_vector(7 downto 0);
53         outputPort : out std_logic_vector(7 downto 0)
54     );
55 end component UsbPort;
56
57     component PELCD is
58     port (
59         SDX, CLK, SS, RESET : in  std_logic;
60         Q                   : out std_logic_vector(9 downto 0)
61     );
62 end component PELCD;
63
64     component PETD is
65     port (
66         SDX, CLK, SS, RESET : in  std_logic;
67         D                   : out std_logic_vector(7 downto 0);
68         Prt, Rt             : out std_logic
69     );
70 end component PETD;
71
72     component TICKET_DISPENSER is
73     port (
74         RT, Prt, CollectTicket : in  std_logic;
75         O, D                   : in  std_logic_vector(3 downto 0);
76         Fn                     : out std_logic;
77         HEX0, HEX1, HEX2, HEX3, HEX4, HEX5 : out std_logic_vector(7 downto 0)
78     );
79 end component TICKET_DISPENSER;
80
81     component CoinAcceptor is
82     port (

```

```

83         Coin          : in  std_logic;
84         CId           : in  std_logic_vector(2 downto 0);
85         CoinInputBits : out std_logic_vector(3 downto 0);
86         AcceptCmd      : in  std_logic;
87         EjectCmd       : in  std_logic;
88         CollectCmd     : in  std_logic;
89         Accept          : out std_logic;
90         Eject           : out std_logic;
91         Collect        : out std_logic
92     );
93 end component CoinAcceptor;
94
95 -----
96 -- Internal signals
97 -----
98
99 -- USB bridge
100 signal INPUT_PORT_LINK      : std_logic_vector(7 downto 0);
101 signal OUTPUT_PORT_LINK    : std_logic_vector(7 downto 0);
102
103 -- Keyboard
104 signal KVAL_LINK, TXD, TXclk : std_logic;
105 signal KEY_CODE_LINK        : std_logic_vector(3 downto 0);
106
107 -- LCD serial frame
108 signal LCD_FRAME_LINK      : std_logic_vector(9 downto 0);
109
110 -- Ticket serial data
111 signal PETD_D_LINK         : std_logic_vector(7 downto 0);
112 signal PRT_LINK            : std_logic;
113 signal RT_LINK             : std_logic;
114
115 -- Ticket dispenser
116 signal FN_LINK, clk_out, clk_out_rise : std_logic;
117 signal COIN_INPUT_LINK      : std_logic_vector(3 downto 0);
118
119 begin
120
121 -----
122 -- USB virtual JTAG port
123 -- INPUT_PORT_LINK byte layout:
124 -- [7]      = Kval (key valid strobe)
125 -- [6]      = '0' (reserved)
126 -- [5]      = '0' (reserved)
127 -- [4]      = Fn   (ticket dispenser finished flag)
128 -- [3..0]   = K    (4-bit key code)
129 -----
130 INPUT_PORT_LINK(4)      <= FN_LINK;
131 INPUT_PORT_LINK(3 downto 0) <= COIN_INPUT_LINK;
132 INPUT_PORT_LINK(7)      <= TXD;
133 TXclk                   <= OUTPUT_PORT_LINK(7);
134
135 USB: component UsbPort
136 port map (
137     inputPort      => INPUT_PORT_LINK,
138     outputPort     => OUTPUT_PORT_LINK
139 );
140
141 -----
142
143 -- Keyboard reader
144 -- Tdelay = "11" -> 2000 ms debounce (same as original)
145 -- Kval and K are exposed so the USB byte can be built above.
146 -- The Kack/KBfree feedback loop is handled inside

```

```

147 — KeyboardReader      no external wiring needed.
148 —
149 — Note: if your KeyboardReader does not expose Kval/K as
150 — top-level ports, move those signals back here and wire
151 — Kack from a local KBfree signal instead.
152 —
153 KBD: component KeyboardReader
154 port map (
155     CLK           => CLK,
156     RESET         => RESET,
157     Tdelay        => Tdelay,
158     Keys_Vertical => Keys_Vertical,
159     Keys_Horizontal => Keys_Horizontal,
160     TXclk         => TXclk,
161     TXD           => TXD,
162     clk_out       => clk_out,
163     clk_out_rise  => clk_out_rise
164 );
165
166 —
167 — LCD serial decoder
168 — Software drives SDX/CLK/SS via output port bits 0,1,2
169 — OUTPUT_PORT_LINK bit assignment (set by software):
170 — [0] = SDX (serial data)
171 — [1] = CLK (serial clock)
172 — [2] = SS (LCD slave select)
173 — [3] = SS (ticket slave select)
174 — [7] = Kack (key acknowledge only used in old design,
175 —           now handled internally by KeyboardReader)
176 —
177 LCD_SERIAL: component PELCD
178 port map (
179     RESET         => RESET,
180     CLK           => OUTPUT_PORT_LINK(1),
181     SDX           => OUTPUT_PORT_LINK(0),
182     SS            => OUTPUT_PORT_LINK(2),
183     Q             => LCD_FRAME_LINK
184 );
185
186 — LCD frame bit assignment (matches original):
187 — Q[9] = RS (register select)
188 — Q[8:1] = DATA (data byte, bit 0 first in frame = Q[8] = DATA[0])
189 — Q[0] = E (enable strobe)
190 LCD_RS          <= LCD_FRAME_LINK(9);
191 LCD_DATA(0)     <= LCD_FRAME_LINK(8);
192 LCD_DATA(1)     <= LCD_FRAME_LINK(7);
193 LCD_DATA(2)     <= LCD_FRAME_LINK(6);
194 LCD_DATA(3)     <= LCD_FRAME_LINK(5);
195 LCD_DATA(4)     <= LCD_FRAME_LINK(4);
196 LCD_DATA(5)     <= LCD_FRAME_LINK(3);
197 LCD_DATA(6)     <= LCD_FRAME_LINK(2);
198 LCD_DATA(7)     <= LCD_FRAME_LINK(1);
199 LCD_EN          <= LCD_FRAME_LINK(0);
200
201 —
202 — Ticket serial decoder
203 — Same SDX/CLK lines as LCD, different slave select (bit 3)
204 —
205 TICKET_SERIAL: component PETD
206 port map (
207     RESET         => RESET,
208     CLK           => OUTPUT_PORT_LINK(1),
209     SDX           => OUTPUT_PORT_LINK(0),
210     SS            => OUTPUT_PORT_LINK(3),

```

```

211         D                => PETD_D_LINK,
212         Rt                => RT_LINK,
213         Prt                => PRT_LINK
214     );
215
216     —————
217     — Ticket dispenser FSM + 7-segment display drivers
218     — PETD_D_LINK byte layout (set by software via PETD):
219     — [3..0] = O (ticket option / quantity low nibble)
220     — [7..4] = D (ticket destination / quantity high nibble)
221     —————
222     TICKET_DISP: component TICKET_DISPENSER
223     port map (
224         RT                => RT_LINK,
225         Prt                => PRT_LINK,
226         CollectTicket     => CollectTicket ,
227         O                  => PETD_D_LINK(3 downto 0) ,
228         D                  => PETD_D_LINK(7 downto 4) ,
229         Fn                 => FN_LINK,
230         HEX0               => HEX0,
231         HEX1               => HEX1,
232         HEX2               => HEX2,
233         HEX3               => HEX3,
234         HEX4               => HEX4,
235         HEX5               => HEX5
236     );
237
238     — Coin Acceptor
239     COIN_ACCEPTOR: component CoinAcceptor
240     port map (
241         Coin               => Coin ,
242         CId                => CId ,
243         CoinInputBits      => COIN_INPUT_LINK,
244         AcceptCmd          => OUTPUT_PORT_LINK(4) ,
245         EjectCmd           => OUTPUT_PORT_LINK(5) ,
246         CollectCmd         => OUTPUT_PORT_LINK(6) ,
247         Accept             => Accept ,
248         Eject              => Eject ,
249         Collect            => Collect
250     );
251
252     INPUT_PORT_LINK(6) <= M;
253
254 end architecture logicFunction;

```

A.2 Atribuição de Pinos

Clock	pino
CLK	P11
Swiches	pino
Tdelay[0]	C10
Tdelay[1]	C11
CId[0]	D12
CId[1]	C12
CId[2]	A12
Coin	B12
CollectTicket	A13
M	B14
RESET	F15
LED's	pino
Accept	A8
Eject	A9
Collect	A10
4x4 Matrix Keypad	pino
Keys_Vertical[0]	AB11
Keys_Vertical[1]	AB10
Keys_Vertical[2]	AA9
Keys_Vertical[3]	AA8
Keys_Horizontal[0]	W5
Keys_Horizontal[1]	AA14
Keys_Horizontal[2]	W12
Keys_Horizontal[3]	AB12
LCD – 8-bit parallel + RS + EN	pino
LCD_RS	W8
LCD_EN	V5
LCD_DATA[0]	AA15
LCD_DATA[1]	W13
LCD_DATA[2]	AB13
LCD_DATA[3]	Y11
LCD_DATA[4]	W11
LCD_DATA[5]	AA10
LCD_DATA[6]	Y8
LCD_DATA[7]	Y7

Tabela 2: Atribuição de pinos do ficheiro TicketMachine.vhd [4]

HEX	Pino
HEX0[0]	C14
HEX0[1]	E15
HEX0[2]	C15
HEX0[3]	C16
HEX0[4]	E16
HEX0[5]	D17
HEX0[6]	C17
HEX0[7]	D15
HEX1[0]	C18
HEX1[1]	D18
HEX1[2]	E18
HEX1[3]	B16
HEX1[4]	A17
HEX1[5]	A18
HEX1[6]	B17
HEX1[7]	A16
HEX2[0]	B20
HEX2[1]	A20
HEX2[2]	B19
HEX2[3]	A21
HEX2[4]	B21
HEX2[5]	C22
HEX2[6]	B22
HEX2[7]	A19

HEX	Pino
HEX3[0]	F21
HEX3[1]	E22
HEX3[2]	E21
HEX3[3]	C19
HEX3[4]	C20
HEX3[5]	D19
HEX3[6]	E17
HEX3[7]	D22
HEX4[0]	F18
HEX4[1]	E20
HEX4[2]	E19
HEX4[3]	J18
HEX4[4]	H19
HEX4[5]	F19
HEX4[6]	F20
HEX4[7]	F17
HEX5[0]	J20
HEX5[1]	K20
HEX5[2]	L18
HEX5[3]	N18
HEX5[4]	M20
HEX5[5]	N19
HEX5[6]	N20
HEX5[7]	L19

Tabela 2: Atribuição de pinos do ficheiro TicketMachine.vhd (continuação)

A.3 Atribuição do USBPort

Import	pino
CoinId(0)	I0
CoinId(1)	I1
CoinId(2)	I2
Coin	I3
Fn	I4
-	I5
M	I6
TXd	I7
Outport	pino
SDX	O0
SCLK	O1
SSLCD	O2
SSTD	O3
accept	O4
eject	O5
collect	O6
TXclk	O7

Tabela 3: Atribuição de variáveis para o USBPort

B Ticket Machine - Software (*Control*)

B.1 Ticket Machine - App

Algoritmo 2: Ticket Machine Application

```

1  import java.time.LocalDateTime
2  import java.time.format.DateTimeFormatter
3
4  private const val STATION_COUNT = 16
5  private const val DIGIT_TIMEOUT_MS = 500L
6  private const val ABORT_DISPLAY_MS = 2000L
7  private const val THANK_YOU_DISPLAY_MS = 5000L
8  private const val DATE_FORMAT = "dd/MM/yyyy_HH:mm"
9  private const val LCD_WIDTH = 16
10 private const val M_MASK = 0b01000000
11
12 private const val MAINT_SCROLL_MS = 1000L
13
14 class App {
15
16     private var currentDestiny = 0
17
18     fun init() {
19         TUI.init()
20         TicketDispenser.init()
21         Stations.init()
22         CoinAcceptor.init()
23     }
24
25     fun start() {
26         while (true) {
27             if (HAL.isBit(M_MASK)) {
28                 maintenanceMode()
29             } else {
30                 showSplashScreen()
31                 currentDestiny = 0
32                 listDestinations()
33             }
34         }
35     }
36
37     private fun maintenanceMode() {
38         val menuItems = listOf(
39             "#-Print_ticket",
40             "A-Station_Count",
41             "B-Coin_Count",
42             "C-Sys_Reset",
43             "D-Shutdown"
44         )
45         var itemIndex = 0
46         var lastScrollTime = System.currentTimeMillis()
47
48         TUI.clear()
49         TUI.cursor(0, centeredPadding("Maintenance".length))
50         TUI.write("Maintenance")
51         renderScrollLine(menuItems[itemIndex])
52
53         while (HAL.isBit(M_MASK)) {
54             val now = System.currentTimeMillis()
55             if (now - lastScrollTime >= MAINT_SCROLL_MS) {
56                 itemIndex = (itemIndex + 1) % menuItems.size
57                 renderScrollLine(menuItems[itemIndex])

```

```

58         lastScrollTime = now
59     }
60
61     when (TUI.getKey()) {
62         '#' -> maintTestPrint()
63         'A' -> maintStationCounters()
64         'B' -> maintCoinCounters()
65         'C' -> maintSystemReset()
66         'D' -> maintShutdown()
67     }
68
69     TUI.cursor(0, centeredPadding("Maintenance".length))
70     TUI.write("Maintenance")
71 }
72
73
74 private fun renderScrollLine(text: String) {
75     TUI.deleteText(LCD.Line.LOWER, 0, LCD_WIDTH - 1)
76     TUI.cursor(1, 0)
77     TUI.write(text)
78 }
79
80 private fun maintTestPrint() {
81     currentDestiny = 0
82     maintSelectDestinationForTest()
83
84     // Restore maintenance header after returning
85     TUI.clear()
86     TUI.cursor(0, centeredPadding("Maintenance".length))
87     TUI.write("Maintenance")
88 }
89
90 private fun maintSelectDestinationForTest() {
91     var firstDigit: Int? = null
92     var digitTime = 0L
93
94     renderDestinationMenu()
95
96     while (true) {
97         if (firstDigit != null &&
98             (System.currentTimeMillis() - digitTime > DIGIT_TIMEOUT_MS ||
99                 firstDigit != 1)
100         ) {
101             currentDestiny = firstDigit
102             firstDigit = null
103             renderDestinationMenu()
104         }
105
106         when (val key = TUI.getKey()) {
107             in '0'..'9' -> {
108                 val digit = key.digitToInt()
109                 if (firstDigit == null) {
110                     firstDigit = digit
111                     digitTime = System.currentTimeMillis()
112                 } else {
113                     val combined = firstDigit * 10 + digit
114                     currentDestiny = if (combined in 0 until STATION_COUNT)
115                         combined else firstDigit!!
116                     firstDigit = null
117                     renderDestinationMenu()
118                 }
119             }
120             'A' -> {
121                 firstDigit = null

```

```

120         currentDestiny = (currentDestiny - 1 + STATION_COUNT) %
121             STATION_COUNT
122         renderDestinationMenu()
123     }
124     'B' -> {
125         firstDigit = null
126         currentDestiny = (currentDestiny + 1) % STATION_COUNT
127         renderDestinationMenu()
128     }
129     '#' -> {
130         firstDigit = null
131         // Enter test-print confirmation screen
132         val station = Stations.getStation(currentDestiny) ?: return
133         maintTestPrintConfirm(station)
134         return
135     }
136 }
137 }
138
139 private fun maintTestPrintConfirm(station: Station) {
140     TUI.clear()
141     TUI.cursor(0, centeredPadding(station.name.length))
142     TUI.write(station.name)
143     TUI.cursor(1, 2)
144     TUI.write("*-to print")
145
146     while (true) {
147         when (TUI.getKey()) {
148             '*' -> {
149                 TUI.deleteText(LCD.Line.LOWER, 0, 15)
150                 TUI.cursor(1, 1)
151                 TUI.write("Collect Ticket")
152                 TicketDispenser.activatePrintingTicket(false, 15, station.id)
153                 TUI.clear()
154                 TUI.cursor(0, 4)
155                 TUI.write("Thank You!")
156                 TUI.cursor(1, 0)
157                 TUI.write("Have a Nice Trip!")
158                 Thread.sleep(THANK_YOU_DISPLAY_MS)
159                 return
160             }
161             '#' -> {
162                 abortVending()
163                 return
164             }
165         }
166     }
167 }
168
169 private fun maintStationCounters() {
170     var idx = 0
171     renderStationCounterScreen(idx)
172
173     while (true) {
174         when (TUI.getKey()) {
175             'A' -> {
176                 idx = (idx - 1 + STATION_COUNT) % STATION_COUNT
177                 renderStationCounterScreen(idx)
178             }
179             'B' -> {
180                 idx = (idx + 1) % STATION_COUNT
181                 renderStationCounterScreen(idx)
182             }
183         }
184     }
185 }

```

```

183         '#' -> return
184     }
185 }
186 }
187
188 private fun renderStationCounterScreen(idx: Int) {
189     val station = Stations.getStation(idx) ?: return
190     TUI.clear()
191     // Line 0: station name centered
192     TUI.cursor(0, centeredPadding(station.name.length))
193     TUI.write(station.name)
194     // Line 1: station id (left, 2 digits) + ticket count (right)
195     TUI.cursor(1, 0)
196     TUI.write(String.format("%02d", station.id))
197     TUI.writeIcon(Icons.UPWARDS_ARROW)
198     TUI.writeIcon(Icons.DOWNWARDS_ARROW)
199     val countStr = station.currentTicketCount.toString()
200     TUI.cursor(1, LCD_WIDTH - countStr.length)
201     TUI.write(countStr)
202 }
203
204 private val coinValues = listOf(5, 10, 20, 50, 100, 200)
205
206 private fun maintCoinCounters() {
207     var idx = 0
208     renderCoinCounterScreen(idx)
209
210     while (true) {
211         when (TUI.getKey()) {
212             'A' -> {
213                 idx = (idx - 1 + coinValues.size) % coinValues.size
214                 renderCoinCounterScreen(idx)
215             }
216             'B' -> {
217                 idx = (idx + 1) % coinValues.size
218                 renderCoinCounterScreen(idx)
219             }
220             '#' -> return
221         }
222     }
223 }
224
225 private fun renderCoinCounterScreen(idx: Int) {
226     val coinCents = coinValues[idx]
227     val label = formatCurrency(coinCents) // e.g. "0.05"
228     val coin = CoinDeposit.storedCoins.getOrNull(idx)
229     val count = coin?.currentCount ?: 0
230
231     TUI.clear()
232     // Line 0: coin value centered
233     TUI.cursor(0, centeredPadding(label.length))
234     TUI.write(label)
235     TUI.writeIcon(Icons.EURO_SIGN)
236     // Line 1: index (2 digits, left) + count (right)
237     TUI.cursor(1, 0)
238     TUI.write(String.format("%02d", idx))
239     TUI.writeIcon(Icons.UPWARDS_ARROW)
240     TUI.writeIcon(Icons.DOWNWARDS_ARROW)
241     val countStr = count.toString()
242     TUI.cursor(1, LCD_WIDTH - countStr.length)
243     TUI.write(countStr)
244 }
245
246 private fun maintSystemReset() {

```

```

247     TUI.clear()
248     TUI.cursor(0, centeredPadding("ResetSystem".length))
249     TUI.write("ResetSystem")
250     TUI.cursor(1, 0)
251     TUI.write("*-YesOther-No")
252     while (true) {
253         when (TUI.getKey()) {
254             '*' -> {
255                 TUI.deleteText(LCD.Line.LOWER, 0, LCD_WIDTH - 1)
256                 TUI.cursor(1, 0)
257                 TUI.write("Resetting...")
258                 reset()
259                 sleep(1500)
260             }
261             '\u0000' -> {}
262             else -> return
263         }
264     }
265 }
266
267 private fun maintShutdown()
268
269 {
270
271     TUI.clear()
272     TUI.cursor(0, centeredPadding("Shutdown?".length))
273     TUI.write("Shutdown?")
274     TUI.cursor(1, 1)
275     TUI.write("*-Yes/Other-No")
276
277     while (true) {
278         when (TUI.getKey()) {
279             '*' -> {
280                 writeFile()
281                 TUI.deleteText(LCD.Line.LOWER, 0, LCD_WIDTH - 1)
282                 TUI.cursor(1, 0)
283                 TUI.write("Shuttingdown...")
284                 sleep(1500)
285                 TUI.clear()
286                 kotlin.system.exitProcess(0)
287             }
288             '\u0000' -> {}
289             else -> return
290         }
291     }
292 }
293
294 private fun showSplashScreen() {
295     TUI.clear()
296     TUI.write("\uTickettoRide")
297     TUI.cursor(1, 0)
298     TUI.write(LocalDateTime.now().format(DateTimeFormatter.ofPattern(DATE_FORMAT)))
299     while (TUI.getKey() == '\u0000' && !HAL.isBit(M_MASK)) {}
300 }
301
302 private fun listDestinations() {
303     if (HAL.isBit(M_MASK)) return
304
305     var firstDigit: Int? = null
306     var digitTime = 0L
307
308     renderDestinationMenu()
309 }
310

```

```

311     while (true) {
312         if (firstDigit != null && System.currentTimeMillis() - digitTime >
313             DIGIT_TIMEOUT_MS || firstDigit != null && firstDigit != 1) {
314             currentDestiny = firstDigit
315             firstDigit = null
316             renderDestinationMenu()
317         }
318         when (val key = TUI.getKey()) {
319             in '0'..'9' -> {
320                 val digit = key.digitToInt()
321                 if (firstDigit == null) {
322                     firstDigit = digit
323                     digitTime = System.currentTimeMillis()
324                 } else {
325                     val combined = firstDigit * 10 + digit
326                     currentDestiny = if (combined in 0 until STATION_COUNT)
327                         combined else firstDigit!!
328                     firstDigit = null
329                     renderDestinationMenu()
330                 }
331             }
332             'A' -> {
333                 firstDigit = null
334                 currentDestiny = (currentDestiny - 1 + STATION_COUNT) %
335                     STATION_COUNT
336                 renderDestinationMenu()
337             }
338             'B' -> {
339                 firstDigit = null
340                 currentDestiny = (currentDestiny + 1) % STATION_COUNT
341                 renderDestinationMenu()
342             }
343             '#' -> {
344                 firstDigit = null
345                 selectDestiny()
346                 return
347             }
348         }
349     }
350     private fun selectDestiny() {
351         val station = Stations.getStation(currentDestiny) ?: return
352
353         var roundTrip = false
354         renderFullDestinationScreen(station, roundTrip)
355
356         while (true) {
357             val remaining = priceFor(station, roundTrip) - CoinDeposit.ammoutInserted()
358
359             if (remaining <= 0) {
360                 dispenseTicket(station, roundTrip)
361                 return
362             }
363
364             if (CoinAcceptor.acceptCoin()) {
365                 if (priceFor(station, roundTrip) - CoinDeposit.ammoutInserted() < 0)
366                     continue
367                 renderRemainingAmount(priceFor(station, roundTrip) - CoinDeposit.
368                     ammoutInserted(), roundTrip, 0)
369             }
370             val f = TUI.getKey()
371             when (f) {

```

```

370         '*' -> {
371             roundTrip = !roundTrip
372             renderRemainingAmount(priceFor(station, roundTrip) - CoinDeposit.
                ammoutInserted(), roundTrip, 0)
373         }
374         '#' -> {
375             abortVending()
376             return
377         }
378     }
379 }
380 }
381
382 private fun dispenseTicket(station: Station, roundTrip: Boolean) {
383     showLoading(2, 0)
384     sleep(1500)
385     showLoading(2, 1)
386     CoinAcceptor.collectCoins()
387     showLoading(2, 2)
388     sleep(1500)
389     showCollectTicket()
390     collectTicket(station, roundTrip)
391 }
392
393 private fun collectTicket(station: Station, roundTrip: Boolean) {
394     Stations.addTicket(station.id)
395     TicketDispenser.activatePrintingTicket(roundTrip, 15, station.id)
396
397     TUI.clear()
398     TUI.cursor(0, 4)
399     TUI.write("Thank You!")
400     TUI.cursor(1, 0)
401     TUI.write("Have a Nice Trip!")
402
403     Thread.sleep(THANK_YOU_DISPLAY_MS)
404 }
405
406 private fun abortVending() {
407     TUI.clear()
408     TUI.write("VENDING ABORTED")
409
410     val inserted = CoinDeposit.ammoutInserted()
411     if (inserted > 0) {
412         TUI.cursor(1, 0)
413         TUI.write("Returning")
414         TUI.write(formatCurrency(inserted))
415         TUI.writeIcon(Icons.EURO_SIGN)
416         CoinAcceptor.ejectCoins()
417     }
418
419     Thread.sleep(ABORT_DISPLAY_MS)
420 }
421
422 private fun showLoading(size: Int, completed: Int) {
423     TUI.deleteText(LCD.Line.LOWER, 0, LCD_WIDTH - 1)
424     val start = centeredPadding(size + 2)
425     TUI.cursor(1, start)
426     TUI.writeIcon(Icons.LEFT_PROGRESSBAR_ICON)
427     repeat(completed) {
428         TUI.writeIcon(Icons.MIDDLE_FULL_PROGRESSBAR_ICON)
429     }
430     repeat(size - completed) {
431         TUI.writeIcon(Icons.MIDDLE_EMPTY_PROGRESSBAR_ICON)
432     }

```

```

433     TUI.writeIcon(Icons.RIGHT_PROGRESSBAR_ICON)
434 }
435
436 private fun showCollectTicket() {
437     TUI.deleteText(LCD.Line.LOWER, 0, 15)
438     TUI.cursor(1, 1)
439     TUI.write("CollectTicket")
440 }
441
442 private fun renderDestinationMenu() {
443     val station = Stations.getStation(currentDestiny) ?: return
444     TUI.clear()
445     renderDestinationRow(station, showId = true)
446     renderRemainingAmount(priceFor(station, false), true, 2)
447 }
448
449 private fun renderFullDestinationScreen(station: Station, roundTrip: Boolean) {
450     TUI.clear()
451     renderDestinationRow(station, showId = false)
452     renderRemainingAmount(priceFor(station, roundTrip) - CoinDeposit.ammoutInserted
453         (), roundTrip, 0)
454 }
455
456 private fun renderDestinationRow(station: Station, showId: Boolean) {
457     TUI.cursor(0, centeredPadding(station.name.length))
458     TUI.write(station.name)
459
460     TUI.cursor(1, 0)
461     if (showId) TUI.write(String.format("%02d", station.id))
462 }
463
464 private fun renderRemainingAmount(remaining: Int, roundTrip: Boolean, startC: Int =
465     1) {
466     TUI.cursor(1, startC)
467     TUI.writeIcon(Icons.UPWARDS_ARROW)
468     if (roundTrip) {
469         TUI.writeIcon(Icons.DOWNWARDS_ARROW)
470     } else {
471         TUI.deleteText(LCD.Line.LOWER, 1, 2)
472     }
473     TUI.cursor(1, 11)
474     TUI.write(formatCurrency(remaining))
475     TUI.writeIcon(Icons.EURO_SIGN)
476 }
477
478 private fun priceFor(station: Station, roundTrip: Boolean): Int =
479     if (roundTrip) station.price * 2 else station.price
480
481 private fun centeredPadding(textLength: Int): Int =
482     (LCD_WIDTH - textLength).coerceAtLeast(0) / 2
483
484 private fun formatCurrency(cents: Int): String =
485     String.format("%.2f", cents / 100.0)
486
487 fun writeFile() {
488     CoinDeposit.writeFile()
489     Stations.writeFile()
490 }
491
492 fun reset() {
493     Stations.resetTicketCounters()
494     CoinDeposit.resetCoinCounters()
495     CoinDeposit.resetCoinDeposit()
496 }

```

495 }

B.2 TUI

Algoritmo 3: TUI

```

1  import java.time.LocalDateTime
2  import java.time.format.DateTimeFormatter
3
4  private const val STATION_COUNT = 16
5  private const val DIGIT_TIMEOUT_MS = 500L
6  private const val ABORT_DISPLAY_MS = 2000L
7  private const val THANK_YOU_DISPLAY_MS = 5000L
8  private const val DATE_FORMAT = "dd/MM/yyyy_HH:mm"
9  private const val LCD_WIDTH = 16
10 private const val M_MASK = 0b01000000
11
12 private const val MAINT_SCROLL_MS = 1000L
13
14 class App {
15
16     private var currentDestiny = 0
17
18     fun init() {
19         TUI.init()
20         TicketDispenser.init()
21         Stations.init()
22         CoinAcceptor.init()
23     }
24
25     fun start() {
26         while (true) {
27             if (HAL.isBit(M_MASK)) {
28                 maintenanceMode()
29             } else {
30                 showSplashScreen()
31                 currentDestiny = 0
32                 listDestinations()
33             }
34         }
35     }
36
37     private fun maintenanceMode() {
38         val menuItems = listOf(
39             "#-Print_ticket",
40             "A-Station_Count",
41             "B-Coin_Count",
42             "C-Sys_Reset",
43             "D-Shutdown"
44         )
45         var itemIndex = 0
46         var lastScrollTime = System.currentTimeMillis()
47
48         TUI.clear()
49         TUI.cursor(0, centeredPadding("Maintenance".length))
50         TUI.write("Maintenance")
51         renderScrollLine(menuItems[itemIndex])
52
53         while (HAL.isBit(M_MASK)) {
54             val now = System.currentTimeMillis()
55             if (now - lastScrollTime >= MAINT_SCROLL_MS) {
56                 itemIndex = (itemIndex + 1) % menuItems.size

```

```

57         renderScrollLine(menuItems[itemIndex])
58         lastScrollTime = now
59     }
60
61     when (TUI.getKey()) {
62         '#' -> maintTestPrint()
63         'A' -> maintStationCounters()
64         'B' -> maintCoinCounters()
65         'C' -> maintSystemReset()
66         'D' -> maintShutdown()
67     }
68
69     TUI.cursor(0, centeredPadding("Maintenance".length))
70     TUI.write("Maintenance")
71 }
72
73
74 private fun renderScrollLine(text: String) {
75     TUI.deleteText(LCD.Line.LOWER, 0, LCD_WIDTH - 1)
76     TUI.cursor(1, 0)
77     TUI.write(text)
78 }
79
80 private fun maintTestPrint() {
81     currentDestiny = 0
82     maintSelectDestinationForTest()
83
84     // Restore maintenance header after returning
85     TUI.clear()
86     TUI.cursor(0, centeredPadding("Maintenance".length))
87     TUI.write("Maintenance")
88 }
89
90 private fun maintSelectDestinationForTest() {
91     var firstDigit: Int? = null
92     var digitTime = 0L
93
94     renderDestinationMenu()
95
96     while (true) {
97         if (firstDigit != null &&
98             (System.currentTimeMillis() - digitTime > DIGIT_TIMEOUT_MS ||
99              firstDigit != 1)
100         ) {
101             currentDestiny = firstDigit
102             firstDigit = null
103             renderDestinationMenu()
104         }
105
106         when (val key = TUI.getKey()) {
107             in '0'..'9' -> {
108                 val digit = key.digitToInt()
109                 if (firstDigit == null) {
110                     firstDigit = digit
111                     digitTime = System.currentTimeMillis()
112                 } else {
113                     val combined = firstDigit * 10 + digit
114                     currentDestiny = if (combined in 0 until STATION_COUNT)
115                         combined else firstDigit!!
116                     firstDigit = null
117                     renderDestinationMenu()
118                 }
119             }
120             'A' -> {

```

```

119         firstDigit = null
120         currentDestiny = (currentDestiny - 1 + STATION_COUNT) %
            STATION_COUNT
121         renderDestinationMenu()
122     }
123     'B' -> {
124         firstDigit = null
125         currentDestiny = (currentDestiny + 1) % STATION_COUNT
126         renderDestinationMenu()
127     }
128     '#' -> {
129         firstDigit = null
130         // Enter test-print confirmation screen
131         val station = Stations.getStation(currentDestiny) ?: return
132         maintTestPrintConfirm(station)
133         return
134     }
135     }
136 }
137
138
139 private fun maintTestPrintConfirm(station: Station) {
140     TUI.clear()
141     TUI.cursor(0, centeredPadding(station.name.length))
142     TUI.write(station.name)
143     TUI.cursor(1, 2)
144     TUI.write("*-to print")
145
146     while (true) {
147         when (TUI.getKey()) {
148             '*' -> {
149                 TUI.deleteText(LCD.Line.LOWER, 0, 15)
150                 TUI.cursor(1, 1)
151                 TUI.write("Collect Ticket")
152                 TicketDispenser.activatePrintingTicket(false, 15, station.id)
153                 TUI.clear()
154                 TUI.cursor(0, 4)
155                 TUI.write("Thank You!")
156                 TUI.cursor(1, 0)
157                 TUI.write("Have a Nice Trip!")
158                 Thread.sleep(THANK_YOU_DISPLAY_MS)
159                 return
160             }
161             '#' -> {
162                 abortVending()
163                 return
164             }
165         }
166     }
167 }
168
169 private fun maintStationCounters() {
170     var idx = 0
171     renderStationCounterScreen(idx)
172
173     while (true) {
174         when (TUI.getKey()) {
175             'A' -> {
176                 idx = (idx - 1 + STATION_COUNT) % STATION_COUNT
177                 renderStationCounterScreen(idx)
178             }
179             'B' -> {
180                 idx = (idx + 1) % STATION_COUNT
181                 renderStationCounterScreen(idx)

```

```

182         }
183         '#' -> return
184     }
185 }
186 }
187
188 private fun renderStationCounterScreen(idx: Int) {
189     val station = Stations.getStation(idx) ?: return
190     TUI.clear()
191     // Line 0: station name centered
192     TUI.cursor(0, centeredPadding(station.name.length))
193     TUI.write(station.name)
194     // Line 1: station id (left, 2 digits) + ticket count (right)
195     TUI.cursor(1, 0)
196     TUI.write(String.format("%02d", station.id))
197     TUI.writeIcon(Icons.UPWARDS_ARROW)
198     TUI.writeIcon(Icons.DOWNWARDS_ARROW)
199     val countStr = station.currentTicketCount.toString()
200     TUI.cursor(1, LCD_WIDTH - countStr.length)
201     TUI.write(countStr)
202 }
203
204 private val coinValues = listOf(5, 10, 20, 50, 100, 200)
205
206 private fun maintCoinCounters() {
207     var idx = 0
208     renderCoinCounterScreen(idx)
209
210     while (true) {
211         when (TUI.getKey()) {
212             'A' -> {
213                 idx = (idx - 1 + coinValues.size) % coinValues.size
214                 renderCoinCounterScreen(idx)
215             }
216             'B' -> {
217                 idx = (idx + 1) % coinValues.size
218                 renderCoinCounterScreen(idx)
219             }
220             '#' -> return
221         }
222     }
223 }
224
225 private fun renderCoinCounterScreen(idx: Int) {
226     val coinCents = coinValues[idx]
227     val label = formatCurrency(coinCents) // e.g. "0.05"
228     val coin = CoinDeposit.storedCoins.getOrNull(idx)
229     val count = coin?.currentCount ?: 0
230
231     TUI.clear()
232     // Line 0: coin value centered
233     TUI.cursor(0, centeredPadding(label.length))
234     TUI.write(label)
235     TUI.writeIcon(Icons.EURO_SIGN)
236     // Line 1: index (2 digits, left) + count (right)
237     TUI.cursor(1, 0)
238     TUI.write(String.format("%02d", idx))
239     TUI.writeIcon(Icons.UPWARDS_ARROW)
240     TUI.writeIcon(Icons.DOWNWARDS_ARROW)
241     val countStr = count.toString()
242     TUI.cursor(1, LCD_WIDTH - countStr.length)
243     TUI.write(countStr)
244 }
245

```

```

246     private fun maintSystemReset() {
247         TUI.clear()
248         TUI.cursor(0, centeredPadding("Reset_System".length))
249         TUI.write("Reset_System")
250         TUI.cursor(1, 0)
251         TUI.write("*_Yes_Other-No")
252         while (true) {
253             when (TUI.getKey()) {
254                 '*' -> {
255                     TUI.deleteText(LCD.Line.LOWER, 0, LCD_WIDTH - 1)
256                     TUI.cursor(1, 0)
257                     TUI.write("Resetting...")
258                     reset()
259                     sleep(1500)
260                 }
261                 '\u0000' -> {}
262                 else -> return
263             }
264         }
265     }
266 }
267
268 private fun maintShutdown()
269
270 {
271     TUI.clear()
272     TUI.cursor(0, centeredPadding("Shutdown?".length))
273     TUI.write("Shutdown?")
274     TUI.cursor(1, 1)
275     TUI.write("*-Yes/Other-No")
276
277     while (true) {
278         when (TUI.getKey()) {
279             '*' -> {
280                 writeFile()
281                 TUI.deleteText(LCD.Line.LOWER, 0, LCD_WIDTH - 1)
282                 TUI.cursor(1, 0)
283                 TUI.write("Shutting_down...")
284                 sleep(1500)
285                 TUI.clear()
286                 kotlin.system.exitProcess(0)
287             }
288             '\u0000' -> {}
289             else -> return
290         }
291     }
292 }
293
294 private fun showSplashScreen() {
295     TUI.clear()
296     TUI.write("_Ticket_to_Ride")
297     TUI.cursor(1, 0)
298     TUI.write(LocalDate.now().format(DateTimeFormatter.ofPattern("DATE_FORMAT")))
299     while (TUI.getKey() == '\u0000' && !HAL.isBit(M_MASK)) {}
300 }
301
302 private fun listDestinations() {
303     if (HAL.isBit(M_MASK)) return
304
305     var firstDigit: Int? = null
306     var digitTime = 0L
307
308     renderDestinationMenu()
309 }

```

```

310
311     while (true) {
312         if (firstDigit != null && System.currentTimeMillis() - digitTime >
313             DIGIT_TIMEOUT_MS || firstDigit != null && firstDigit != 1) {
314             currentDestiny = firstDigit
315             firstDigit = null
316             renderDestinationMenu()
317         }
318
319         when (val key = TUI.getKey()) {
320             in '0'..'9' -> {
321                 val digit = key.digitToInt()
322                 if (firstDigit == null) {
323                     firstDigit = digit
324                     digitTime = System.currentTimeMillis()
325                 } else {
326                     val combined = firstDigit * 10 + digit
327                     currentDestiny = if (combined in 0 until STATION_COUNT)
328                         combined else firstDigit!!
329                     firstDigit = null
330                     renderDestinationMenu()
331                 }
332             }
333             'A' -> {
334                 firstDigit = null
335                 currentDestiny = (currentDestiny - 1 + STATION_COUNT) %
336                     STATION_COUNT
337                 renderDestinationMenu()
338             }
339             'B' -> {
340                 firstDigit = null
341                 currentDestiny = (currentDestiny + 1) % STATION_COUNT
342                 renderDestinationMenu()
343             }
344             '#' -> {
345                 firstDigit = null
346                 selectDestiny()
347                 return
348             }
349         }
350     }
351
352     private fun selectDestiny() {
353         val station = Stations.getStation(currentDestiny) ?: return
354
355         var roundTrip = false
356         renderFullDestinationScreen(station, roundTrip)
357
358         while (true) {
359             val remaining = priceFor(station, roundTrip) - CoinDeposit.ammoutInserted()
360
361             if (remaining <= 0) {
362                 dispenseTicket(station, roundTrip)
363                 return
364             }
365
366             if (CoinAcceptor.acceptCoin()) {
367                 if (priceFor(station, roundTrip) - CoinDeposit.ammoutInserted() < 0)
368                     continue
369                 renderRemainingAmount(priceFor(station, roundTrip) - CoinDeposit.
370                     ammoutInserted(), roundTrip, 0)
371             }
372             val f = TUI.getKey()

```

```

369         when (f) {
370             '*' -> {
371                 roundTrip = !roundTrip
372                 renderRemainingAmount(priceFor(station, roundTrip) - CoinDeposit.
                    ammoutInserted(), roundTrip, 0)
373             }
374             '#' -> {
375                 abortVending()
376                 return
377             }
378         }
379     }
380 }
381
382 private fun dispenseTicket(station: Station, roundTrip: Boolean) {
383     showLoading(2, 0)
384     sleep(1500)
385     showLoading(2, 1)
386     CoinAcceptor.collectCoins()
387     showLoading(2, 2)
388     sleep(1500)
389     showCollectTicket()
390     collectTicket(station, roundTrip)
391 }
392
393 private fun collectTicket(station: Station, roundTrip: Boolean) {
394     Stations.addTicket(station.id)
395     TicketDispenser.activatePrintingTicket(roundTrip, 15, station.id)
396
397     TUI.clear()
398     TUI.cursor(0, 4)
399     TUI.write("Thank You!")
400     TUI.cursor(1, 0)
401     TUI.write("Have a Nice Trip!")
402
403     Thread.sleep(THANK_YOU_DISPLAY_MS)
404 }
405
406 private fun abortVending() {
407     TUI.clear()
408     TUI.write("VENDING ABORTED")
409
410     val inserted = CoinDeposit.ammoutInserted()
411     if (inserted > 0) {
412         TUI.cursor(1, 0)
413         TUI.write("Returning")
414         TUI.write(formatCurrency(inserted))
415         TUI.writeIcon(Icons.EURO_SIGN)
416         CoinAcceptor.ejectCoins()
417     }
418
419     Thread.sleep(ABORT_DISPLAY_MS)
420 }
421
422 private fun showLoading(size: Int, completed: Int) {
423     TUI.deleteText(LCD.Line.LOWER, 0, LCD_WIDTH - 1)
424     val start = centeredPadding(size + 2)
425     TUI.cursor(1, start)
426     TUI.writeIcon(Icons.LEFT_PROGRESSBAR_ICON)
427     repeat(completed) {
428         TUI.writeIcon(Icons.MIDDLE_FULL_PROGRESSBAR_ICON)
429     }
430     repeat(size - completed) {
431         TUI.writeIcon(Icons.MIDDLE_EMPTY_PROGRESSBAR_ICON)

```

```

432     }
433     TUI.writeIcon(Icons.RIGHT_PROGRESSBAR_ICON)
434 }
435
436 private fun showCollectTicket() {
437     TUI.deleteText(LCD.Line.LOWER, 0, 15)
438     TUI.cursor(1, 1)
439     TUI.write("CollectTicket")
440 }
441
442 private fun renderDestinationMenu() {
443     val station = Stations.getStation(currentDestiny) ?: return
444     TUI.clear()
445     renderDestinationRow(station, showId = true)
446     renderRemainingAmount(priceFor(station, false), true, 2)
447 }
448
449 private fun renderFullDestinationScreen(station: Station, roundTrip: Boolean) {
450     TUI.clear()
451     renderDestinationRow(station, showId = false)
452     renderRemainingAmount(priceFor(station, roundTrip) - CoinDeposit.ammoutInserted
453         (), roundTrip, 0)
454 }
455
456 private fun renderDestinationRow(station: Station, showId: Boolean) {
457     TUI.cursor(0, centeredPadding(station.name.length))
458     TUI.write(station.name)
459
460     TUI.cursor(1, 0)
461     if (showId) TUI.write(String.format("%02d", station.id))
462 }
463
464 private fun renderRemainingAmount(remaining: Int, roundTrip: Boolean, startC: Int =
465     1) {
466     TUI.cursor(1, startC)
467     TUI.writeIcon(Icons.UPWARDS_ARROW)
468     if (roundTrip) {
469         TUI.writeIcon(Icons.DOWNWARDS_ARROW)
470     } else {
471         TUI.deleteText(LCD.Line.LOWER, 1, 2)
472     }
473     TUI.cursor(1, 11)
474     TUI.write(formatCurrency(remaining))
475     TUI.writeIcon(Icons.EURO_SIGN)
476 }
477
478 private fun priceFor(station: Station, roundTrip: Boolean): Int =
479     if (roundTrip) station.price * 2 else station.price
480
481 private fun centeredPadding(textLength: Int): Int =
482     (LCD_WIDTH - textLength).coerceAtLeast(0) / 2
483
484 private fun formatCurrency(cents: Int): String =
485     String.format("%.2f", cents / 100.0)
486
487 fun writeFile() {
488     CoinDeposit.writeFile()
489     Stations.writeFile()
490 }
491
492 fun reset() {
493     Stations.resetTicketCounters()
494     CoinDeposit.resetCoinCounters()
495     CoinDeposit.resetCoinDeposit()

```

494 }
495 }

B.3 Coin Acceptor

Algoritmo 4: Coin Acceptor

```

1  import java.lang.Thread
2
3  object CoinAcceptor {
4      private const val CID_MASK = 0b00000111
5      private const val COIN_MASK = 0b00001000
6
7      private const val COIN_EJECT_MASK = 0b00100000
8      private const val COIN_COLLECT_MASK = 0b01000000
9      private const val COIN_ACCEPT_MASK = 0b00010000
10
11     val coinCode_List: List<Int> = listOf(5, 10, 20, 50, 100, 200, 0, 0)
12
13     fun init() {
14         CoinDeposit.init()
15         HAL.clrBits(COIN_ACCEPT_MASK)
16         HAL.clrBits(COIN_EJECT_MASK)
17         HAL.clrBits(COIN_COLLECT_MASK)
18     }
19
20     fun hasCoin(): Boolean {
21         return HAL.isBit(COIN_MASK)
22     }
23
24     fun getCoinID(): Int? {
25         if (hasCoin()) {
26             val value = HAL.readBits(CID_MASK)
27
28             if (value in 0 until 6) {
29                 return coinCode_List[value]
30             }
31         }
32
33         return null
34     }
35
36     fun acceptCoin(): Boolean {
37         if (hasCoin()) {
38             val c = getCoinID()
39             HAL.setBits(COIN_ACCEPT_MASK)
40
41             if (c != null) CoinDeposit.add(c)
42             while (hasCoin());
43
44             HAL.clrBits(COIN_ACCEPT_MASK)
45             return true
46         }
47         return false
48     }
49
50     fun ejectCoins() {
51         HAL.setBits(COIN_EJECT_MASK)
52         CoinDeposit.ejectInsertedCoins()
53         Thread.sleep(2000L)
54         HAL.clrBits(COIN_EJECT_MASK)
55     }

```

```

56
57     fun collectCoins() {
58         HAL.setBits(COIN_COLLECT_MASK)
59         CoinDeposit.collectStoredCoins()
60         Thread.sleep(2000L)
61         HAL.clrBits(COIN_COLLECT_MASK)
62     }
63 }

```

B.4 Coin Deposit

Algoritmo 5: Coin Deposit

```

1  data class Coin(val type: Int, var currentCount: Int, val id: Int)
2
3  object CoinDeposit {
4
5      var storedCoins: MutableList<Coin> = mutableListOf()
6      var insertedCoins: MutableList<Coin> = mutableListOf()
7
8
9      fun init() {
10         loadStoredCoins()
11         ejectInsertedCoins()
12     }
13
14     private fun loadStoredCoins() {
15         readFile("CoinDeposit.txt").forEachIndexed { index, line ->
16             val (type, count) = line.split(";")
17             storedCoins.add(Coin(type.toInt(), count.toInt(), index))
18             insertedCoins.add(Coin(type.toInt(), 0, index))
19         }
20     }
21
22     fun add(type: Int) {
23         insertedCoins.find { it.type == type }?.let { it.currentCount++ }
24     }
25
26     fun collectStoredCoins() {
27         storedCoins.zip(insertedCoins).forEach { (stored, inserted) ->
28             stored.currentCount += inserted.currentCount
29         }
30         resetCoinCounters()
31     }
32
33     fun ammountInserted(): Int {
34         return insertedCoins.sumOf { it.type * it.currentCount }
35     }
36
37     fun ejectInsertedCoins() {
38         insertedCoins.forEach { it.currentCount = 0 }
39     }
40
41     fun resetCoinCounters() {
42         insertedCoins.forEach { it.currentCount = 0 }
43     }
44
45     fun resetCoinDeposit() {
46         storedCoins.forEach { it.currentCount = 0 }
47     }
48
49     fun writeFile() {

```

```

50     val output = storedCoins.map { "${it.type};${it.currentCount}" }
51     writeFile(fileName = "CoinDeposit.txt", dataArray = output.toTypedArray())
52 }
53 }

```

B.5 Stations

Algoritmo 6: Stations

```

1  data class Station(val price: Int, var currentTicketCount: Int, val name: String, val
   id: Int)
2
3  object Stations {
4
5      var stationsInfo: MutableList<Station> = mutableListOf()
6
7      fun init() {
8          loadStationsInfo()
9      }
10
11     private fun loadStationsInfo() {
12         stationsInfo = readFile("stations.txt")
13             .mapIndexed { index, line ->
14                 val (price, count, name) = line.split(";")
15                 Station(price.toInt(), count.toInt(), name, index)
16             }.toMutableList()
17     }
18
19     fun getStation(stationID: Int): Station? = stationsInfo.find { it.id == stationID }
20
21
22     fun addTicket(stationID: Int) {
23         stationsInfo.find { it.id == stationID }?.let { it.currentTicketCount++ }
24     }
25
26     fun resetTicketCounters() {
27         stationsInfo.forEach { it.currentTicketCount = 0 }
28     }
29
30     fun writeFile() {
31         val output = stationsInfo.map { "${it.price};${it.currentTicketCount};${it.name}
32             }" }
33         writeFile(fileName = "stations.txt", dataArray = output.toTypedArray())
34     }

```

B.6 File Access

Algoritmo 7: File Access

```

1  import java.io.BufferedReader
2  import java.io.FileReader
3  import java.io.PrintWriter
4
5  fun readFile(fileName: String): Array<String> {
6      val reader = BufferedReader(FileReader(fileName))
7
8      var dataArray: Array<String> = emptyArray()
9

```

```
10     var currentLine = reader.readLine()
11     while (currentLine != null) {
12         dataArray += currentLine
13         currentLine = reader.readLine()
14     }
15
16     reader.close()
17     return dataArray
18 }
19
20 fun writeFile(fileName: String, dataArray: Array<String>) {
21     val writer = PrintWriter(fileName)
22     for (data in dataArray) {
23         writer.println(data)
24     }
25     writer.close()
26 }
```